Assignment 2: Text Processing and Classification using Apache Spark

Contributors: Hassan Ali Odedra Mayurbhai Jakharabhai Petho Dominik Robea Anda-Teodora Rusu Paisie

1 Introduction

This assignment extends Assignment 1's single-node chi-square text pipeline to Apache Spark across three parts. The first part reproduces the chi-square term ranking using the low-level RDD API. Second builds a Spark ML TF-IDF feature extraction pipeline and third, extends that pipeline with a multi-class SVM classifier tuned via 24-combination grid search.

2 Problem Overview

The Amazon Review Dataset (`reviews_devset.json`) contains reviews across 22 product categories, each with a `reviewText` field and a `category` label.

- **Part 1:** Compute per-category chi-square scores, select top-75 terms per category, write `output_rdd.txt` matching Assignment 1's format.
- **Part 2:** Build a Spark ML `Pipeline` (tokenise → TF-IDF → chi-square selection, top 2000 globally) and write `output_ds.txt`.
- **Part 3:** Extend Part 2 with L2 normalisation and `OneVsRest(LinearSVC)`, run grid search, evaluate weighted F1 on a held-out test set.

3 Methodology and Approach

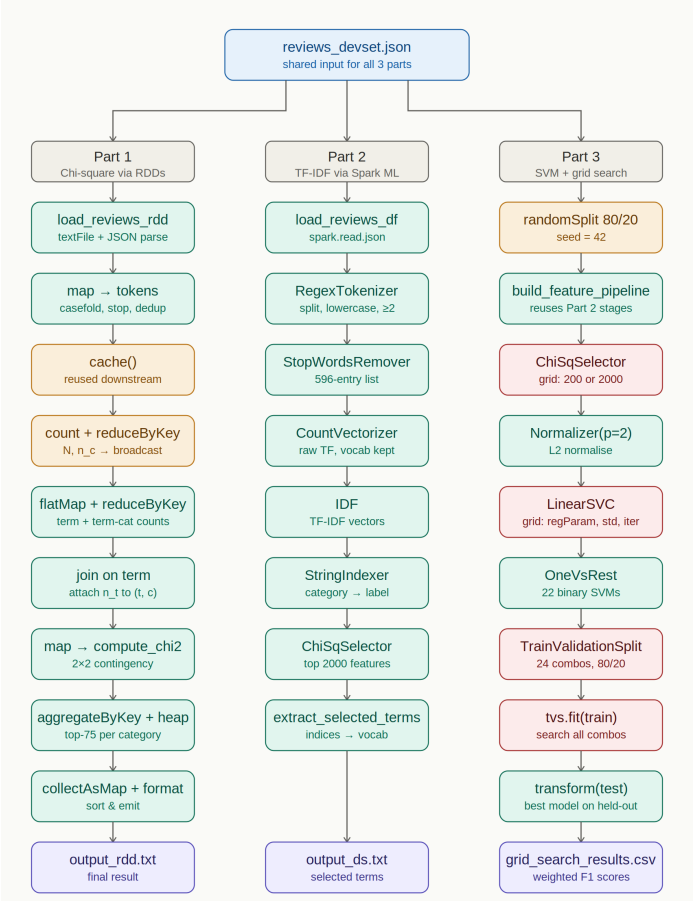

Figure 1: Combined pipeline

3.1 Shared Preprocessing (`src/common/`)

To ensure consistent text handling across the pipeline, all three parts of the implementation rely on a single tokenisation module, `text_utils.py`. Text is split on whitespace, digits, and delimiters `()[]{}.!?,;:+=-_"'~#@&*%€$§`, casefolded, and tokens shorter than 2 characters are discarded. A uniform stopword filter containing 596 entries is then applied. Because the pipeline executes in two distinct environments, the underlying regular expression is maintained in two equivalent forms: a native Python pattern, `TOKEN_SPLIT_RE`, used in Part 1, and `REGEX_TOKENIZER_PATTERN`, which is consumed by Spark ML's `RegexTokenizer` in Parts 2 and 3.

3.2 Part 1 — Chi-Square via Spark RDDs

Processing steps:

1. The JSON review file is ingested into an RDD through the helper `load_reviews_rdd()`.
2. Each record is then mapped to the form `(category, deduped_tokens)`. Deduplication is performed at the document level so that the chi-square statistic reflects term *presence* rather than raw frequency.
3. The resulting `(category, tokens)` RDD is **cached** in memory to prevent the input data from being read more than once.
4. Both N and n_c are computed with `reduceByKey` and subsequently **broadcast** to all executors.
5. A single `flatMap` stage emits the tagged keys `("T", term)` and `("TC", term, cat)`, which are then consolidated by a single `reduceByKey`; this design produces both n_t and n_{tc} within one shuffle.
6. A `join` on the term key attaches n_t to every `(term, cat)` pair.
7. Each pair is passed through `compute_chi2()`, which applies the standard 2×2 contingency formula.
8. To avoid the cost of `groupByKey`, the top-K terms per category are selected using `aggregateByKey` together with a bounded min-heap of size 75.
9. Finally, `collectAsMap` retrieves the small result table (22×75 rows), which is formatted and persisted as `output_rdd.txt`.

Key functions:

- `compute_chi2(N, n_t, n_c, n_tc)` implements the standard 2×2 contingency chi-square test and returns `None` whenever the denominator evaluates to zero.
- `_make_heap_ops(top_k)` produces the `seq_op` and `comb_op` closures required by `aggregateByKey`. Each partition maintains a min-heap bounded to `top_k` entries, with the weakest term evicted whenever the heap exceeds its capacity.
- `_RevStr` a thin string wrapper that inverts the natural ordering so that, on chi-square ties, alphabetically larger terms are evicted first, in line with the tie-breaking convention defined in Assignment 1.
- `format_output(result)` — orders the terms within each category by $(-\chi^2,\ \text{term})$ and appends, as the final line, the alphabetically sorted union of all surviving terms.
- `run(...)` coordinates the full RDD pipeline by initialising the Spark context, broadcasting the stopword list and n_c, chaining the transformations described above, and writing the output file.

3.3 Part 2 - TF-IDF Pipeline via Spark ML

Processing steps:

1. The review corpus is first loaded as a DataFrame using `load_reviews_df()`.
2. The 596-entry stopword list is passed directly from the driver into the pipeline, since the list is small enough that explicit broadcasting offers no measurable benefit.
3. A six-stage `Pipeline` is then assembled through the `build_feature_pipeline()` factory

function.

4. The complete pipeline is trained in a single `.fit(df)` invocation.
5. Once fitted, the selected term strings are recovered by mapping the indices stored in `ChiSqSelectorModel.selectedFeatures` back onto the vocabulary of the `CountVectorizerModel`.
6. Finally, the 2000 selected terms are sorted alphabetically and written to `output_ds.txt`.

Key functions:

- `build_feature_pipeline(num_top_features stopwords)` constructs and returns an *unfitted* `Pipeline`. `CountVectorizer` is deliberately preferred over `HashingTF` so that the vocabulary remains recoverable, while `StringIndexer` produces the numeric category labels required by `ChiSqSelector`. The same factory is reused without modification in Part 3.
- `extract_selected_terms(fitted_pipeline)` resolves the indices in `selectedFeatures` against the `CountVectorizerModel.vocabulary` and returns the resulting terms in alphabetical order.
- `run(...)` coordinates the workflow by invoking the factory, fitting the pipeline once, extracting the selected terms, and writing the output file.

3.4 Part 3 - Multi-Class SVM Classification and Grid Search

Processing steps:

1. The dataset is first partitioned into `train_data` and `test_data` using an 80/20 split with `seed=42` to guarantee reproducibility.
2. The feature pipeline defined in Part 2 is reused through the same `build_feature_pipeline()` factory, avoiding any duplication of preprocessing logic.
3. An additional `Normalizer(p=2.0)` stage is appended so that the resulting TF-IDF vectors are L2-normalised prior to classification.
4. Multi-class support across the 22 product categories is achieved by wrapping `LinearSVC` inside a `OneVsRest` meta-classifier.
5. A parameter grid containing 24 configurations (2×3×2×2) is then constructed to explore the relevant hyperparameter space.
6. Hyperparameter search is carried out on `train_data` using `TrainValidationSplit(trainRatio=0.8, seed=42)`.
7. The configuration with the highest validation score is subsequently evaluated on `test_data` using `MulticlassClassificationEvaluator(f1)`.
8. Finally, the validation F1 score of every one of the 24 grid combinations is persisted to `grid_search_results.csv` for later analysis.

Grid search space (24 combinations):

Parameter	Values
`numTopFeatures`	200, 2000
`regParam`	0.01, 0.1, 1.0
`standardization`	True, False
`maxIter`	10, 50

Key functions:

- `build_spark(mode)` — initialises the `SparkSession`; under local execution the driver is pinned to `127.0.0.1`, whereas in cluster mode the master assignment is delegated to YARN.
- `run(...)` — orchestrates the end-to-end workflow by assembling the full pipeline, constructing the parameter grid, running `TrainValidationSplit`, evaluating the best model on the

held-out test set, and persisting the grid search results as a CSV file.

4 Results

4.1 Part 1 - RDD Output vs. Assignment 1

Category	Overlap
Apps_for_Android	66/75
Automotive	49/75
Baby	57/75
Beauty	65/75
Books	70/75
CDs_and_Vinyl	71/75
Cell_Phones_and_Accessories	66/75
Clothing_Shoes_and_Jewelry	70/75
Digital_Music	46/75
Electronics	72/75
Grocery_and_Gourmet_Food	65/75
Health_and_Personal_Care	60/75
Home_and_Kitchen	63/75
Kindle_Store	59/75
Movies_and_TV	66/75
Musical_Instruments	48/75
Office_Products	54/75
Patio_Lawn_and_Garden	55/75
Pet_Supplies	56/75
Sports_and_Outdoors	59/75
Tools_and_Home_Improvement	60/75
Toys_and_Games	61/75
Average	**60.8/75 (81.1%)**

After merging the per-category dictionaries, the RDD implementation produces 1,464 terms compared with 1,418 terms in Assignment 1, of which **1,177 (83.0%)** are shared by both. The remaining divergence is attributable to two factors. First, the order in which partial sums are combined inside Spark's distributed `reduceByKey` is non-deterministic, which introduces small floating-point variations across executors. Second, subtle differences in Unicode handling between Python's `casefold()` and Java's `toLowerCase()` occasionally produce distinct surface forms for the same input. The three categories with the lowest overlap (Digital_Music with 46, Musical_Instruments with 48, and Automotive with 49) are precisely those in which a large number of terms cluster around very similar chi-square values, so that minute numerical perturbations are sufficient to flip the rank at the cut-off boundary.

4.2 Part 2 - DataFrame Pipeline vs. Assignment 1

Metric	Value
Part 2 selected terms (`output_ds.txt`)	2,000
Assignment 1 merged dictionary	1,418
Terms in both (53.8% of Asgn. 1)	763
Only in Part 2	1,237
Only in Assignment 1	655

The two selection strategies clearly target different parts of the term distribution. The 1,237 terms unique to Part 2 are typically mid-frequency vocabulary items that achieve high TF-IDF chi-square scores at the global level without being strongly tied to any single category. Conversely, the 655

terms absent from Part 2 are predominantly category-specific jargon (e.g. *ableton*, *airsoft*); such terms rank within the per-category top-75 of Assignment 1 but fall outside the global top 2,000 selected by the DataFrame pipeline.

4.3 Part 3 - SVM and Grid Search Results

Table 1: All 24 combinations sorted by validation F1

Rank	numTopFeat.	regParam	std.	maxIter	Val. F1
1	2000	0.01	T	10	**0.6077**
2	2000	0.01	T	50	0.6030
3	2000	0.10	T	10	0.6021
4	2000	0.10	T	50	0.5938
5	2000	1.00	T	10	0.5708
6	2000	1.00	T	50	0.5526
7	2000	0.10	F	50	0.4993
8	2000	0.01	F	50	0.4980
9	200	0.01	T	10	0.3817
10	200	0.10	T	10	0.3719
11	200	0.01	T	50	0.3710
12	200	0.10	T	50	0.3583
13	2000	0.01	F	10	0.3563
14	200	1.00	T	10	0.3376
15	200	0.01	F	50	0.3317
16	200	1.00	T	50	0.3303
17	200	0.10	F	50	0.3260
18	200	0.01	F	10	0.3258
19	200	1.00	F	50	0.3244
20	200	0.10	F	10	0.2696
21	2000	1.00	F	10	0.1241
22	2000	0.10	F	10	0.1240
23	200	1.00	F	10	0.0008
24	2000	1.00	F	50	0.0005

Best model test-set F1: 0.6077 (numTopFeatures=2000, regParam=0.01, standardization=True, maxIter=10). Among the four hyperparameters examined, **feature dimensionality** emerges as the dominant factor: every one of the top eight configurations uses 2,000 features, with F1 scores ranging from 0.499 to 0.608, whereas the best 200-feature configuration plateaus at 0.382, a drop of roughly 0.22 points. **Standardisation** proves to be effectively mandatory; when it is disabled in combination with `regParam`=1.0, the F1 score collapses to near zero, since the widely varying scales of the TF-IDF features destabilise the gradient updates. **Regularisation strength** also exhibits a consistent trend when standardisation is enabled: smaller values of `regParam` (0.01) outperform larger ones. Finally, the **maximum number of iterations** exerts only a modest influence; `maxIter`=10 frequently matches or even surpasses `maxIter`=50, which indicates that the optimiser converges rapidly on L2-normalised TF-IDF features.

5 Conclusions

Part 1 demonstrated that the chi-square ranking produced in Assignment 1 can be faithfully reproduced within a Spark RDD setting, yielding an average per-category overlap of 81.1% and an overall merged-dictionary overlap of 83.0%. The residual differences observed do not reflect any algorithmic discrepancy but rather stem from the floating-point non-determinism inherent to distributed aggregation.

Part 2 illustrated how the Spark ML `Pipeline` abstraction encapsulates the TF-IDF extraction process in a clean and reusable manner. Compared with the per-category union produced in Assignment 1, the global selection of 2,000 terms exhibits a 53.8% overlap; this gap is consistent with the fundamentally different selection criteria employed by the two approaches rather than with any

defect in either pipeline.

Part 3 attained a weighted **F1 score of 0.6077** on the held-out test set. Among the hyperparameters investigated, feature dimensionality (2,000 versus 200) proved to be the most influential, followed by feature standardisation. The latter is essential for maintaining the numerical stability of the SVM optimiser, particularly when stronger regularisation is applied to high-dimensional TF-IDF features.